

Guía Breve de Enumeraciones (Enum) en Java

Índice Temático

1. Introducción a los Tipos Enumerados	1
1.1. ¿Por qué usar Enums?	1
2. Sintaxis Básica	2
2.1. Uso en sentencias de control	2
3. Enums con Atributos y Métodos	3
3.1. Ejemplo: Planetas y Gravedad	3
4. Métodos Heredados de java.lang.Enum	4
5. Guía de Buenas Prácticas y Buen Uso	5
6. Apéndice: Ejemplos de Programas con Enums	6
6.1. 1. Lógica de Calendario (Días Laborales)	6
6.2. 2. Física Planetaria (Cálculo de Peso)	7
6.3. 3. Gestión Segura de Estados	8
6.4. 4. Roles de Usuario y Permisos	9
6.5. 5. Conversor de Unidades	10
7. Bibliografía y Recursos	10

1. Introducción a los Tipos Enumerados

Una enumeración (**enum**) en Java es un tipo de dato especial que permite que una variable sea un conjunto de constantes predefinidas. La variable debe ser igual a uno de los valores que se han definido para ella.

A diferencia de otros lenguajes donde los enums son simplemente alias para enteros, en Java son clases de pleno derecho que extienden implícitamente la clase `java.lang.Enum`.

1.1. ¿Por qué usar Enums?

Antes de Java 5, la práctica habitual era usar constantes enteras:

```
public static final int LUNES = 1;
public static final int MARTES = 2;
// ...
```

Este enfoque carecía de **seguridad de tipos** (type safety), ya que cualquier entero podía ser pasado a un método que esperara un día de la semana. Los enums resuelven este problema.

2. Sintaxis Básica

La declaración de un enum es similar a la de una clase, pero usando la palabra clave `enum`.

```
public enum DiaSemana {  
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO  
}
```

2.1. Uso en sentencias de control

El uso más común de un enum es en sentencias `switch`, donde Java ofrece una sintaxis muy limpia:

```
DiaSemana hoy = DiaSemana.VIERNES;  
  
switch (hoy) {  
    case LUNES:  
        System.out.println("Comienzo de semana");  
        break;  
    case VIERNES:  
        System.out.println("Casi es fin de semana");  
        break;  
    case SABADO: case DOMINGO:  
        System.out.println("¡Fin de semana!");  
        break;  
    default:  
        System.out.println("Día laboral intermedio");  
}
```

3. Enums con Atributos y Métodos

Un `enum` en Java no es simplemente una lista de etiquetas; es una **clase especializada**. Esto significa que puede contener estado (atributos) y comportamiento (métodos), lo que permite encapsular toda la lógica relacionada con una constante dentro de la misma.

Al ser una clase, podemos definir: * **Atributos (Campos)**: Para almacenar datos específicos de cada constante (ej. el código de un color). * **Constructor**: Para inicializar dichos atributos. **Importante**: El constructor de un enum es siempre privado o de paquete; no se puede invocar con `new`. * **Métodos**: Para procesar los datos de la constante o exponer información al exterior.

3.1. Ejemplo: Planetas y Gravedad

Imaginemos un enum que no solo lista planetas, sino que conoce su física:

```
public enum Planeta {
    MERCURIO (3.303e+23, 2.4397e6),
    VENUS    (4.869e+24, 6.0518e6),
    TIERRA   (5.976e+24, 6.37814e6);

    private final double masa;    // en kilogramos
    private final double radio;   // en metros

    Planeta(double masa, double radio) {
        this.masa = masa;
        this.radio = radio;
    }

    public double gravedadEnSuperficie() {
        final double G = 6.67300E-11;
        return G * masa / (radio * radio);
    }
}
```

4. Métodos Heredados de java.lang.Enum

Todos los enums heredan automáticamente métodos útiles de la clase base:

1. **values()**: Devuelve un array con todas las constantes en el orden en que fueron declaradas.
2. **valueOf(String name)**: Devuelve la constante cuyo nombre coincide exactamente con el string pasado.
3. **name()**: Devuelve el nombre de la constante como String.
4. **ordinal()**: Devuelve la posición de la constante (empezando por 0).

```
for (Planeta p : Planeta.values()) {  
    System.out.printf("La gravedad en %s es %f%n", p.name(), p.  
gravedadEnSuperficie());  
}
```

5. Guía de Buenas Prácticas y Buen Uso

Para utilizar las enumeraciones de forma profesional en Java, se recomienda seguir estas directrices:

1. **Seguridad de Tipos sobre "Constantes Mágicas":** Nunca utilice `int` o `String` para representar un conjunto fijo de opciones.
 - **¿Por qué?** Si un método recibe un `int dia`, nada impide que alguien pase el valor `-5` o `999`. Usando un enum `DiaSemana`, el compilador garantiza que solo se reciban valores válidos definidos en la enumeración. Además, se evitan errores de escritura habituales con strings (ej. "LUNES" vs "Lunes").
2. **Inmutabilidad Absoluta:** Defina siempre los atributos de un enum como `final`. Las constantes de un enum no deben cambiar su estado interno una vez creadas.
3. **Evite el uso de `ordinal()` para Lógica de Negocio:** El método `ordinal()` devuelve la posición de la constante, la cual puede cambiar si alguien reordena el código. Si necesita un valor numérico (como un código de error o un peso), cree un atributo específico.
4. **Encapsule el Comportamiento:** Si un método depende exclusivamente del valor del enum, defínalo dentro del enum. Esto evita tener bloques `switch` dispersos por todo el proyecto (reduciendo el acoplamiento).
5. **Sobrescriba `toString()` si es necesario:** Mientras que `name()` devuelve el identificador técnico, `toString()` puede proporcionar una representación amigable para el usuario final.
6. **Uso de Singleton:** Si necesita una clase de la que solo exista una instancia en toda la aplicación (como un gestor de configuración), el uso de un enum con una sola constante es la implementación más segura y recomendada.

CAUTION

Aunque los enums son muy potentes, no abuse de ellos para conjuntos de datos que cambian con frecuencia (como una lista de clientes). Los enums deben representar conceptos que son conocidos y estables en tiempo de compilación.

6. Apéndice: Ejemplos de Programas con Enums

A continuación se presentan varios ejemplos que demuestran las ventajas de usar enums frente a otras alternativas, incluyendo su código de prueba y salida esperada.

6.1. 1. Lógica de Calendario (Días Laborales)

Uso de enums para clasificar datos de forma semántica y añadir lógica de negocio simple.

```
public enum DiaSemana {
    LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO;

    public boolean esFinDeSemana() {
        return this == SABADO || this == DOMINGO;
    }

    public String tipoDeDia() {
        return esFinDeSemana() ? "Festivo" : "Laboral";
    }
}

public class TestCalendario {
    public static void main(String[] args) {
        for (DiaSemana dia : DiaSemana.values()) {
            System.out.println(dia + " es un día " + dia.tipoDeDia());
        }
    }
}
```

Salida por consola:

```
LUNES es un día Laboral
MARTES es un día Laboral
...
SABADO es un día Festivo
DOMINGO es un día Festivo
```

6.2. 2. Física Planetaria (Cálculo de Peso)

Uso de enums para encapsular fórmulas físicas y constantes universales de forma segura.

```
public enum Planeta {
    MERCURIO(3.303e23, 2.4397e6),
    VENUS(4.869e24, 6.0518e6),
    TIERRA(5.976e24, 6.37814e6),
    MARTE(6.421e23, 3.3972e6);

    private final double masa;    // kg
    private final double radio;   // m
    private static final double G = 6.67300e-11;

    Planeta(double masa, double radio) {
        this.masa = masa;
        this.radio = radio;
    }

    public double gravedadSuperficie() {
        return G * masa / (radio * radio);
    }

    public double pesoEnSuperficie(double masaPersona) {
        return masaPersona * gravedadSuperficie();
    }
}

public class TestPlanetas {
    public static void main(String[] args) {
        double pesoEnTierra = 75.0; // kg
        double masa = pesoEnTierra / Planeta.TIERRA.gravedadSuperficie();

        System.out.println("Peso en la Tierra: " + pesoEnTierra + " kg");
        for (Planeta p : Planeta.values()) {
            if (p != Planeta.TIERRA) {
                System.out.printf("Tu peso en %s sería %.2f kg%n",
                                   p.name(), p.pesoEnSuperficie(masa));
            }
        }
    }
}
```

Salida por consola:

```
Peso en la Tierra: 75.0 kg
Tu peso en MERCURIO sería 28.35 kg
Tu peso en VENUS sería 67.87 kg
Tu peso en MARTE sería 28.25 kg
```

6.3. 3. Gestión Segura de Estados

Este ejemplo muestra cómo un enum evita estados inválidos y centraliza la lógica de transición en un flujo de negocio.

```
public enum EstadoPedido {
    NUEVO, PROCESANDO, ENVIADO, ENTREGADO, CANCELADO;

    public boolean puedeCancelar() {
        return this == NUEVO || this == PROCESANDO;
    }
}

public class TestPedidos {
    public static void main(String[] args) {
        EstadoPedido p1 = EstadoPedido.NUEVO;
        EstadoPedido p2 = EstadoPedido.ENVIADO;

        System.out.println("Pedido 1 (" + p1 + ") ¿puede cancelar?: " + p1
            .puedeCancelar());
        System.out.println("Pedido 2 (" + p2 + ") ¿puede cancelar?: " + p2
            .puedeCancelar());
    }
}
```

Salida por consola:

```
Pedido 1 (NUEVO) ¿puede cancelar?: true
Pedido 2 (ENVIADO) ¿puede cancelar?: false
```


6.4. 4. Roles de Usuario y Permisos

Uso de atributos para encapsular lógica de autorización de forma inmutable y centralizada.

```
public enum RolUsuario {
    ADMIN(true, true), EDITOR(false, true), LECTOR(false, false);

    private final boolean puedeBorrar;
    private final boolean puedeEditar;

    RolUsuario(boolean borrar, boolean editar) {
        this.puedeBorrar = borrar;
        this.puedeEditar = editar;
    }

    public String getInfo() {
        return "Permisos -> Borrar: " + puedeBorrar + ", Editar: " + puedeEditar;
    }
}

public class TestRoles {
    public static void main(String[] args) {
        for (RolUsuario rol : RolUsuario.values()) {
            System.out.println("ROL: " + rol + " | " + rol.getInfo());
        }
    }
}
```

Salida por consola:

```
ROL: ADMIN | Permisos -> Borrar: true, Editar: true
ROL: EDITOR | Permisos -> Borrar: false, Editar: true
ROL: LECTOR | Permisos -> Borrar: false, Editar: false
```

6.5. 5. Conversor de Unidades

Centralización de constantes matemáticas y lógica de conversión dentro de un tipo seguro.

```
public enum UnidadLongitud {
    METRO(1.0), KILOMETRO(1000.0), MILIMETRO(0.001);

    private final double factorAEnMetros;

    UnidadLongitud(double factor) { this.factorAEnMetros = factor; }

    public double convertirA(double valor, UnidadLongitud destino) {
        return (valor * this.factorAEnMetros) / destino.factorAEnMetros;
    }
}

public class TestConversor {
    public static void main(String[] args) {
        double kms = 5.5;
        double metros = UnidadLongitud.KILOMETRO.convertirA(kms, UnidadLongitud.
METRO);
        double mms = UnidadLongitud.METRO.convertirA(1, UnidadLongitud.MILIMETRO);

        System.out.println(kms + " km son " + metros + " metros");
        System.out.println("1 metro son " + mms + " milímetros");
    }
}
```

Salida por consola:

```
5.5 km son 5500.0 metros
1 metro son 1000.0 milímetros
```

7. Bibliografía y Recursos

- **Oracle Java Documentation:** [Enum Types \(The Java Tutorials\)](#)
- **Baeldung:** [A Guide to Java Enums](#)
- **Refactoring Guru:** [Implementación de Singleton con Enum](#)

Los enums son una herramienta poderosa. Si necesita un conjunto de constantes, use un enum. Si necesita que esas constantes tengan datos o comportamiento asociado, los enums en Java están preparados para ello.

— Joshua Bloch